



Industrial Internship Report on

"Password Manager"

Prepared by

[MD.JAMSHADH]

[23M61A0541]

Executive Summary

This report presents the details of the Industrial Internship offered by Upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was based on a project/problem statement provided by UCT, to be completed along with the report within 6 weeks.

My project was a Password Manager built in Python - a desktop application that securely stores, generates and retrieves account passwords using strong encryption, with a graphical interface built using Tkinter and an SQLite database for storage.

This internship gave me a valuable opportunity to gain exposure to a real, industry-relevant problem - protecting sensitive user data - and to design and implement a working solution end to end. It was an overall enriching and practical learning experience.

TABLE OF CONTENTS

1	Preface	Error! Bookmark not defined.
2	Introduction	4-8
2.1	About UniConverge Technologies Pvt Ltd	4-5
2.2	About upskill Campus	5-6
2.3	Objective	6
2.4	Reference	7-8
2.5	Glossary	8
3	Problem Statement	9
4	Existing and Proposed solution	9-10
5	Proposed Design/ Model	10-17
5.1	High Level Diagram (if applicable)	12-14
5.2	Low Level Diagram (if applicable)	14-16
5.3	Interfaces (if applicable)	16-17
6	Performance Test	18-22
6.1	Test Plan/ Test Cases	19
6.2	Test Procedure	20
6.3	Performance Outcome	20-22
7	My learnings	22-28
8	Future work scope	28-30

1. Preface

Over the course of this six-week internship, I worked on designing and building a complete Password Manager application in Python, from the initial idea through to a working, tested product.

An internship like this is an important step in career development because it moves learning out of the classroom and into a real, self-contained project with a defined scope, deadline and deliverable - much like an assignment in a professional setting.

The problem statement given by UCT was to design and build a Password Manager: an application that securely stores a user's account credentials, generates strong passwords, and retrieves them on demand, using encryption to keep the stored data safe.

Upskill Campus and UCT structured this internship around weekly milestones - understanding the problem, designing the architecture, implementing core encryption and storage logic, building the user interface, testing, and finally documenting the work - which gave the six weeks a clear sense of progress.

Through this internship I learned to translate a short problem statement into a working system: choosing an encryption approach, designing a database schema, structuring a Python project into clean modules, building a graphical interface with Tkinter, and writing automated tests to verify the security-critical parts of the code.

I would like to thank Upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd for structuring this internship and providing the problem statement, and my mentors for their guidance throughout the six weeks.

To my juniors and peers about to start a similar internship: start by clearly understanding the problem statement before writing any code, keep your logic and interface code separate from day one, and write a few tests early - they make every change afterwards much less stressful.

2.Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013 and works in the Digital Transformation domain. The company provides industrial solutions with a prime focus on sustainability and Return on Investment (RoI).

UCT works with a range of modern technologies and technical areas. These include Internet of Things (IoT), Cyber Security, Cloud Computing platforms such as AWS and Azure, Machine Learning, communication technologies such as 4G, 5G and LoRaWAN, Java Full Stack development, Python, and front-end development. The range of technologies associated with the organisation provides an industry-oriented environment in which students can gain exposure to different areas of software and technology development.

The internship project provided an opportunity to work on a problem related to cybersecurity and secure application development. The Password Manager problem is particularly relevant to this area because it involves protecting sensitive user information and applying security principles during application development.

The project also demonstrated that software development is not limited to writing program code. A complete application requires planning, architecture design, database selection, security considerations, user-interface design, testing, debugging, and documentation. Working on the project helped connect these different activities into a single development process.

The problem statement provided during the internship required the development of a Password Manager that could securely store passwords, generate strong passwords, and retrieve stored credentials when required. The project was therefore designed with both functionality and security in mind.

The internship environment also encouraged an understanding of how a defined technical problem can be converted into a working software solution. Starting from the problem statement, the project was divided into smaller components and implemented progressively. This approach helped in understanding the importance of breaking a larger software problem into manageable modules.

2.2 About Upskill Campus (USC)

Upskill Campus, along with The IoT Academy and UniConverge Technologies, facilitated the execution of the internship program. Upskill Campus is a career-development platform that focuses on providing personalised, scalable, and measurable career-development opportunities.

The internship provided a structured environment in which the project could be completed within a defined period. Instead of focusing only on theoretical concepts, the internship involved practical implementation through a project-based approach.

The Password Manager project required the application of programming and software-development knowledge to solve a practical problem. During the project, different stages such as understanding requirements, designing the system, implementing individual modules, testing functionality, and preparing documentation were considered.

One of the important benefits of the internship structure was the opportunity to understand the relationship between different areas of software development. For example, database design had to be considered together with security requirements because the application was intended to store sensitive information. Similarly, the graphical interface had to communicate with the business logic without directly accessing sensitive implementation details.

The internship also helped develop problem-solving skills. When developing an application, unexpected errors can occur in areas such as database operations, user input, encryption, or interface behaviour. Identifying the cause of such issues and testing the solution is an important part of practical software development.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT and conducts long-term executive certification programs in collaboration with EICT Academy, IITK, IITR, and IITG across multiple domains.

As part of the internship arrangement, The IoT Academy contributed to the structured learning environment through which the project was carried out. The internship provided an opportunity to work on a defined project while gaining practical exposure to software-development concepts.

The Password Manager project provided exposure to Python application development, cybersecurity concepts, database management, graphical user-interface development, and software testing. These areas are interconnected in the final application because the user interface, database, encryption system, and business logic need to work together correctly.

The project also demonstrated the importance of learning beyond individual programming syntax. For example, knowing Python syntax alone is not sufficient to develop a secure password manager. The developer must also understand how data is stored, how sensitive information is protected, how user input is handled, and how the application can be tested.

The internship therefore provided an opportunity to develop a more complete understanding of application development, from the initial problem statement to the final working product and documentation.

2.4 Objectives of this Internship program

The primary objective of the internship was to gain practical experience in developing a complete software application based on a real-world problem statement. The Password Manager project was selected as the practical implementation task and required both functional and security-related considerations.

The major objectives were:

- To gain practical experience of working on an industry-oriented software project.
- To understand the process of converting a problem statement into a working application.
- To improve practical knowledge of Python programming.
- To understand the basic principles involved in secure application development.
- To learn how encryption can be used to protect sensitive information.
- To understand password-based key derivation using PBKDF2.
- To understand authenticated encryption through the use of Fernet.
- To learn how SQLite can be integrated with a Python desktop application.
- To understand and implement CRUD operations.
- To develop a graphical interface using Tkinter.
- To implement a secure password-generation mechanism.
- To understand the importance of strong and unique passwords.
- To learn how parameterised database queries can be used during database operations.
- To develop modular and maintainable software.
- To write automated unit tests for important application functions.
- To perform manual testing of the graphical interface.
- To identify and handle incorrect user inputs and security-related situations.
- To improve debugging and problem-solving skills.
- To understand the importance of documenting a software project.
- To gain confidence in independently approaching an open-ended technical problem.

The internship also aimed to improve professional skills such as planning, communication, time management, and systematic problem solving. Completing the project within the defined internship period required the work to be divided into stages and completed progressively.

[1] Python documentation - <https://docs.python.org/3/>

[2] cryptography library documentation - <https://cryptography.io/>

[3] Python sqlite3 module documentation - <https://docs.python.org/3/library/sqlite3.html>

2.5 Glossary

Term	Meaning
Master password	The single password the user creates to unlock the vault. It is never stored.
Encryption key	A secret value derived from the master password, used to encrypt and decrypt stored data.
PBKDF2	Password-Based Key Derivation Function 2 - turns a password into a strong encryption key.
Fernet	A symmetric encryption method (AES + HMAC) used to encrypt and authenticate stored data.
Salt	Random data mixed into the master password before key derivation, so the same password never produces the same key twice.
SQLite	A lightweight, file-based database engine used to store the encrypted entries.
Tkinter	Python's built-in library for building desktop graphical user interfaces.
CRUD	Create, Read, Update, Delete - the four basic operations on stored data.

3.Problem Statement

Most people reuse a small number of weak, easy-to-remember passwords across many online accounts. This makes it easy for an attacker who compromises one account or one leaked password list to gain access to several others belonging to the same person.

The assigned problem statement was to design and build a Password Manager: a Python application that allows a user to securely store the passwords for their various accounts, generate strong random passwords, and retrieve stored passwords when needed - all protected behind a single master password, with the stored data encrypted so it is unreadable without that master password.

4.Existing and Proposed solution

Several commercial password managers already exist, such as browser-built-in password managers, and dedicated tools like Bitwarden, KeePass and LastPass. These are mature products, but they also depend on a browser account, a paid subscription, or a third-party cloud service, and they do not expose their internal design - which makes them unsuitable as a learning project even though they are useful as end-user tools.

The proposed solution for this internship is a self-contained, local, open Python application with no cloud dependency and no subscription, where every part of the design - the encryption, the storage and the interface - is understandable and verifiable by reading the code.

The value addition of this project, compared to simply using an existing tool, is that it demonstrates practical, from-scratch understanding of:

- Deriving a strong encryption key from a user-chosen password using PBKDF2.
- Authenticated encryption (Fernet: AES + HMAC) so tampering with stored data is detected, not silently accepted.

- Safe use of a local SQLite database with parameterised queries, to avoid SQL injection.
- Cryptographically secure random password generation using Python's `secrets` module rather than the weaker `random` module.
- A layered software design: encryption, storage and business logic are kept fully separate from the Tkinter user interface, so any one layer can be tested or replaced without touching the others.

Code submission (Github link)

<https://github.com/jamshadh19-999/upskillcampus/blob/main/main.py>

Report submission (Github Link)

5. Proposed Design/ Model

The proposed Password Manager is designed as a secure, modular, and user-friendly desktop application developed using Python. The main purpose of the design is to securely store account credentials, generate strong passwords, and allow users to retrieve and manage their saved information through a graphical interface.

The application follows a **layered architecture** in which different responsibilities are separated into different components. This approach makes the system easier to develop, test, maintain, and modify. The three major parts of the proposed design are the **Core Layer, User Interface Layer, and Data Layer**.

1. Core Layer

The Core Layer contains the main functional and security-related components of the application. It is responsible for encryption, database communication, password generation, and overall password-management operations.

The `crypto.py` module is responsible for deriving the encryption key from the user's master password and performing encryption and decryption operations. PBKDF2 is used for deriving the encryption key, while Fernet is used for authenticated encryption.

The `database.py` module manages communication with the SQLite database. It performs operations such as inserting, retrieving, updating, searching, and deleting password entries. Database queries are parameterised to handle user input safely.

The `generator.py` module generates strong random passwords using Python's `secrets` module. It also provides password-strength evaluation functionality so that users can understand the relative strength of their passwords.

The `manager.py` module acts as the central business-logic component. It connects the encryption, database, and password-generation modules and provides a controlled interface for the graphical user interface.

2. User Interface Layer

The User Interface Layer is developed using Python's Tkinter library. It provides the graphical environment through which the user interacts with the Password Manager.

The `login_frame.py` module manages vault creation and unlocking. The `vault_frame.py` module displays saved password entries and provides options for searching and managing them. The `dialogs.py` module contains different pop-up windows used for adding or editing entries, generating passwords, and changing the master password.

The `app.py` module manages the main application window and security-related functions such as automatic locking and clipboard clearing. The `theme.py` module centralises the colours, fonts, and other interface-related settings.

This separation ensures that the interface does not directly access the database or encryption implementation. Instead, it communicates with the `PasswordManager` class through defined operations.

3. Data Layer

The Data Layer consists of a local SQLite database used for storing the vault information. The database contains two main tables: `meta` and `entries`.

The `meta` table stores information required for vault management, including the salt and verification value. The `entries` table contains information such as service name, username, encrypted password, encrypted notes, and timestamps.

Sensitive information is encrypted before being stored in the database. Therefore, the database does not directly contain the user's passwords in readable form.

5.1 High Level Diagram

The high-level diagram represents the overall working flow of the Password Manager application. The user first opens the application and enters the master password. The master password is used to derive a secure encryption key through the PBKDF2 key derivation process. After successful verification, the user is allowed to access the password vault.

Once the vault is unlocked, the user can perform different operations such as adding a new account, viewing saved accounts, searching for an account, editing existing details, deleting

an entry, and generating a strong password. The generated or entered password is encrypted before it is stored in the SQLite database. Sensitive notes are also encrypted before storage.

When the user requests a saved password, the application retrieves the encrypted information from the database and decrypts it only after the vault has been successfully unlocked. The decrypted password can then be displayed or copied to the clipboard. The application also provides security features such as automatic vault locking, clipboard clearing, and protection against repeated incorrect master-password attempts.

The major components represented in the high-level design are the User Interface, Password Manager/Core Layer, Encryption Module, Password Generator, SQLite Database, and Security Controls. The user interacts only with the graphical interface, while the interface communicates with the PasswordManager class. This separation helps keep the encryption and database operations independent from the user interface.

Figure 1 below shows the high-level flow of the application, from the user entering their master password through to a decrypted password being displayed or copied.

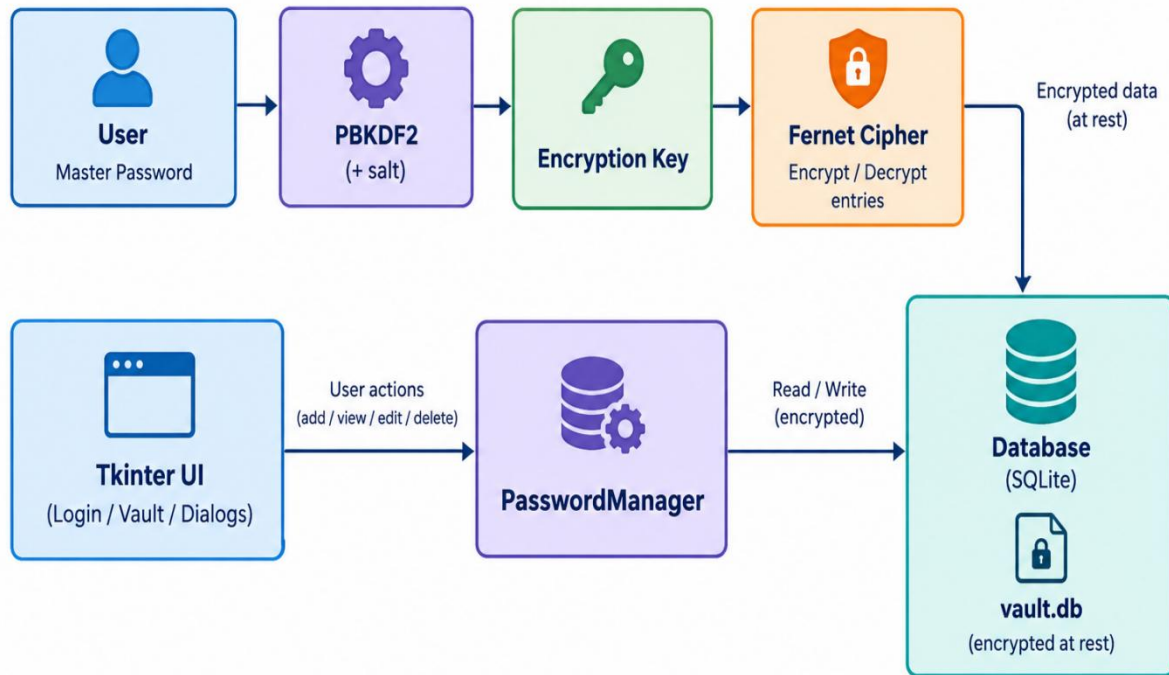


Figure 1: High-level architecture of the Password Manager

5.2 Low Level Diagram

The low-level diagram describes the internal flow and interaction between the individual modules of the Password Manager application. The application is divided into separate modules so that each component performs a specific responsibility.

The login module is responsible for creating the vault and unlocking an existing vault. When the user provides the master password, the crypto module uses the stored salt and PBKDF2 to derive the encryption key. The derived key is then used by the Fernet encryption mechanism for encryption and decryption operations.

The database module manages communication with the SQLite database. It performs operations such as inserting new entries, retrieving saved entries, updating existing records, searching entries, and deleting records. Parameterised SQL queries are used when communicating with the database so that user input is handled safely.

The generator module creates strong random passwords using the Python secrets module. It allows passwords to be generated according to requirements such as length and character types. The password strength functionality also provides an indication of whether a password is weak or strong.

The manager module acts as the central component between the user interface and the core modules. It provides operations such as `add_entry`, `get_entry`, `list_entries`, `update_entry`, `delete_entry`, `unlock`, `lock`, and `change_master_password`. The graphical interface communicates with this manager instead of directly accessing the database or encryption functions.

The user interface contains the login frame, vault frame, dialogs, application window, and theme components. The vault frame displays the stored account information and allows the user to perform operations through the graphical interface. This modular structure makes the application easier to understand, test, maintain, and extend.

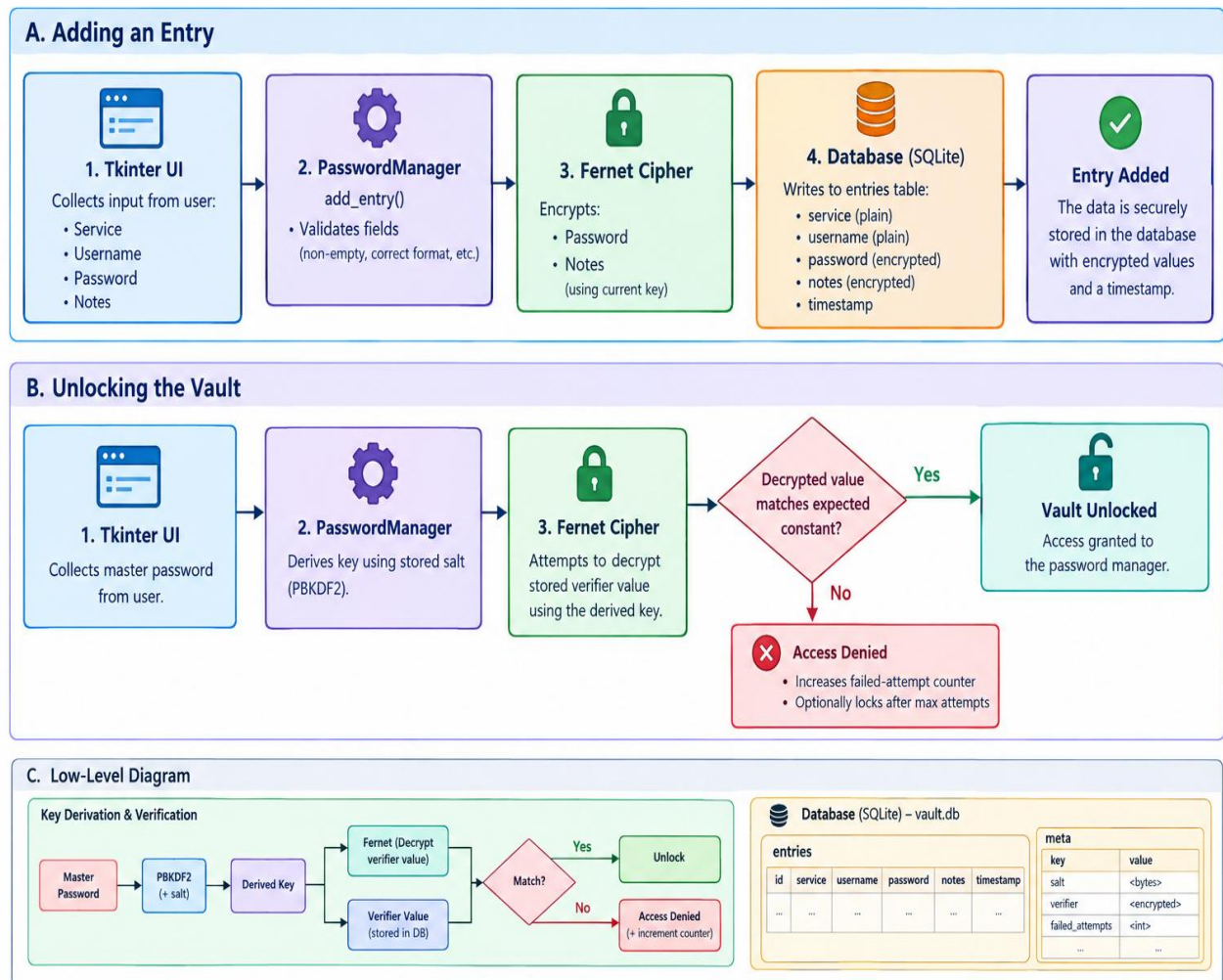


Figure 2: Flow of Adding an Entry and Unlocking the Vault (with Low-Level Diagram)

5.3 Interfaces

The Password Manager provides a graphical desktop interface developed using Python Tkinter. The interface is designed to provide users with a simple way to create, unlock, and manage their password vault without requiring knowledge of the internal encryption or database operations.

The **Login Interface** is used when the application is opened. It allows the user to create a new vault or unlock an existing vault by entering the master password. The master password is not stored directly in the application database.

The **Vault Interface** is the main screen displayed after successful authentication. It provides access to the stored password entries and allows the user to search, view, add, edit, and delete account information. The interface communicates with the PasswordManager class rather than directly accessing the database.

The **Add/Edit Entry Interface** allows users to enter information such as the service name, username, password, and notes. Before sensitive information is stored, it is processed by the core encryption functionality.

The **Password Generator Interface** allows users to generate strong random passwords according to selected requirements such as password length and character types. A password-strength indicator also helps the user understand the strength of a password.

The application also provides interfaces for changing the master password and performing other vault-management operations. Security-related functions such as automatic locking and clipboard clearing work in the background without requiring the user to manually perform these operations.

Internally, the interface communicates with a small set of functions provided by the PasswordManager class, including `add_entry`, `get_entry`, `list_entries`, `update_entry`, `delete_entry`, `unlock`, `lock`, and `change_master_password`. This design ensures that the graphical interface does not directly interact with encryption or database implementation details. PasswordManager_Internship_Repo...

6. Performance Test

Performance testing was carried out to verify the correctness, reliability, security, and responsiveness of the Password Manager application. Since the application handles sensitive information, testing focused not only on speed but also on ensuring that encryption, database operations, password generation, and security controls work correctly.

The application was tested using automated unit tests written with Python's unittest framework. A total of 26 automated tests were created to verify the major functions of the system. These tests covered encryption and decryption, password generation, password-strength checking, database operations, master-password handling, and data persistence.

Encryption tests verified that encrypted data could be successfully decrypted using the correct key. They also verified that the same plaintext does not always produce identical ciphertext. Tests were performed to confirm that incorrect master passwords and modified ciphertext are rejected correctly.

The password generator was tested with different password lengths and character-type combinations. The tests verified that the generated password follows the requested requirements and that special cases, such as digits-only passwords, are handled correctly.

Database performance and reliability were tested using CRUD operations. Entries were added, listed, searched, retrieved, updated, and deleted to confirm that the database behaves correctly. Additional tests verified that the stored data remains available after closing and reopening the application.

Security-related behaviour was also tested. The application automatically locks the vault after five minutes of inactivity, clears copied passwords from the clipboard after twenty seconds, and introduces a thirty-second cooldown after five consecutive incorrect master-password attempts.

For testing, temporary SQLite database files were used so that the tests did not affect the actual user vault. A reduced PBKDF2 iteration count was used during automated tests so that the complete test suite could execute quickly while still testing the application logic.

For a security-focused local application like this one, the two constraints that matter most are correctness of the encryption/decryption logic (an incorrect implementation could silently corrupt or expose data) and reasonable responsiveness of the key-derivation step, since PBKDF2 is deliberately slow.

6.1 Test Plan/ Test Cases

26 automated unit tests were written using Python's unittest framework, covering:

- Encryption round-trips correctly, and the same plaintext produces different ciphertext each time.
- A wrong master password, or tampered ciphertext, is correctly rejected.
- Different salts always produce different keys, even from the same password.
- The password generator always respects the requested length and character-type options, including edge cases such as digits-only passwords.
- The password strength meter correctly ranks weak passwords below strong ones.
- Full CRUD operations on entries: add, list, search, get, update and delete.
- Data persists correctly across an application restart (closing and reopening the vault).
- Changing the master password re-encrypts every stored entry, and the old master password stops working immediately afterwards.
- Search input is treated literally, so a search term containing SQL wildcard characters (% , _) does not behave unexpectedly, and a value containing SQL syntax cannot corrupt the database.

6.2 Test Procedure

The tests were run using the command:

```
python -m unittest discover -v
```

Each test uses a temporary, throwaway SQLite database file so that tests never interfere with a real vault or with each other, and a reduced PBKDF2 iteration count so the full suite runs in well under a second.

6.3 Performance Outcome

The performance outcome of the Password Manager was evaluated through automated unit testing and manual testing of the graphical interface. The main focus of testing was to verify the correctness of encryption and decryption, database operations, password generation, security controls, and overall application responsiveness.

A total of **26 automated unit tests** were implemented using Python's unittest framework. The tests covered encryption round-trips, ciphertext generation, incorrect master-password handling, tampered ciphertext detection, salt-based key derivation, password generation, password-strength checking, and complete database CRUD operations. PasswordManager_Internship_Repo...

The application was also tested for data persistence. Entries were created, modified, and deleted, and the application was restarted to verify that the stored information remained available correctly. Changing the master password was also tested to confirm that the stored entries were re-encrypted and that the previous master password could no longer be used.

Manual testing of the graphical interface confirmed that account entries could be **added, searched, edited, deleted, and copied** correctly. The security controls were also verified. The vault automatically locks after five minutes of inactivity, the clipboard is cleared after

twenty seconds, and five consecutive incorrect master-password attempts result in a thirty-second cooldown period.

SecureVault

Welcome back - enter your master password

Master password

☐ Show password

Unlock

SecureVault Change master password Lock

Search

[+ Add](#) [Edit](#) [Delete](#) [Copy password](#) [Copy username](#) [Password generator](#)

Service / Website	Username / Email	Last updated
Amazon	DavidPaul21@gmail.com	2026-09-23 20:05
College Portal	21B81A0512	2026-09-23 20:08
Flipkart	DavidPaul21@gmail.com	2026-09-23 20:09
Github	David-paul	2026-09-23 20:00
Gmail	DavidPaul21@gmail.com	2026-09-23 20:00
Instagram	David_21	2026-09-23 20:05
LinkedIn	David@gmail.com	2026-09-23 20:03
Netflix	David.A21@gamil.com	2026-09-23 20:07
Phonelock	david	2026-09-23 20:12
phonepe	David	2026-09-23 20:12
Zoom	David.Paul21@gmail.com	2026-09-23 20:10

11 saved accounts | Auto-locks after 5 min of inactivity

7. Learnings

This internship gave me hands-on experience with several concepts that are difficult to learn from theory alone:

- Practical application of cryptography - deriving keys safely with PBKDF2 and using authenticated encryption (Fernet) instead of inventing a custom, likely insecure scheme.
- Secure-by-default coding habits, such as always using parameterised SQL queries and never storing a master password anywhere.
- Structuring a Python project into independent, testable layers (core logic vs. user interface), instead of writing everything inside the UI event handlers.
- Writing automated unit tests for security-sensitive logic, which gave me confidence that changes to the code did not silently break encryption or data storage.

- Building a usable desktop interface with Tkinter, including modal dialogs, input validation and visual feedback such as a password-strength meter.

These skills - particularly secure coding practices, modular software design and automated testing - are directly transferable to any future software development role, and have given me more confidence in approaching an open-ended problem statement independent. During the requirement analysis stage, I learned that a security-oriented application should begin with clearly defined trust boundaries. In this project, the master password is the main secret known to the user, while the stored vault is treated as protected data. This distinction helped me understand why the master password should not be saved directly in the database. Instead, the application derives cryptographic material from it and uses that material to protect the stored information. Working through this requirement made the difference between ordinary application data and security-sensitive data much clearer to me.

Another important learning was the need to think about incorrect input as part of normal application behaviour. A password manager cannot assume that users will always enter correct values. The application therefore has to handle incorrect master passwords, empty fields, invalid lengths, duplicate or unusual search text, and attempts to access entries before the vault is unlocked. Designing these cases made me more careful about validation and error handling.

The internship also improved my ability to break a broad problem statement into smaller tasks. Instead of treating the password manager as one large program, I considered authentication, encryption, database operations, password generation, interface behaviour and testing as separate responsibilities. This made the implementation easier to understand and gave each part a clear purpose.

A major learning outcome from this project was understanding how password-based key derivation fits into an application. A user-selected password is convenient to remember, but it is not automatically suitable as an encryption key. PBKDF2 provides a controlled way of

deriving a key from the master password while using a salt and repeated computation. The salt ensures that identical passwords do not automatically produce identical derived values.

I also learned why authenticated encryption is important. Encryption alone focuses on confidentiality, while authentication helps detect whether protected information has been modified. The project uses Fernet through the cryptography library, allowing the application to encrypt data and reject invalid or tampered ciphertext. This was valuable because it showed me that security is not just about hiding information; it also includes detecting invalid data.

This experience changed the way I think about implementing security features. Rather than inventing an encryption algorithm or combining low-level cryptographic operations without sufficient knowledge, I learned to use established library primitives and follow their documented interfaces. I also learned that security decisions should be made deliberately and documented clearly so that another developer can understand the assumptions behind the implementation.

The database portion of the project taught me how persistence and security have to work together. SQLite was selected because the application is local and does not require a separate database server. The database stores metadata such as the salt and verification information, along with the encrypted fields belonging to each saved entry. The important principle is that sensitive passwords are not stored as ordinary readable database values.

Working with SQLite also improved my understanding of CRUD operations. Adding an account creates a new record, listing and searching retrieve records, updating changes an existing record, and deleting removes a record. These operations look simple individually, but they must be connected correctly to encryption and decryption. For example, a password should be encrypted before it is written and decrypted only when the application has successfully unlocked the vault.

I also learned why parameterised SQL queries matter. User-provided search terms and account information should be treated as values rather than being concatenated into SQL statements. This protects the database layer from SQL injection problems and also makes the code easier to reason about. The tests covering wildcard characters and SQL-like input helped reinforce this lesson.

The password-generation part of the project gave me practical experience with secure random values. A password generator should not simply produce predictable strings or depend on a general-purpose random function when security is required. The project uses Python's secrets module for generating password characters, which is designed for security-sensitive random values.

I learned that password generation is also a user-experience problem. A generated password needs to follow the requested length and character requirements while remaining convenient to copy and use. The strength meter provides additional feedback so that users can distinguish weak passwords from stronger choices. Testing edge cases, including restricted character sets and requested lengths, helped me understand why a generator should validate its inputs instead of assuming that every combination is valid.

This part of the project also showed me how small utility functions can have a large effect on the overall application. A password generator may appear to be a minor feature compared with encryption and storage, but poor randomness or incorrect length handling would reduce the practical value of the password manager. The experience encouraged me to test individual utility functions independently before connecting them to the graphical interface.

Building the Tkinter interface was an important learning experience because it connected the technical core of the application with actual user interaction. The interface includes vault creation and unlocking, a table of stored entries, dialogs for adding and editing records, password generation, and master-password management. I learned that a graphical interface should expose the functionality of the application without exposing unnecessary implementation details.

Separating the interface from the core layer was especially useful. The UI calls the PasswordManager API instead of directly performing database queries or encryption operations. This separation means that interface changes can be made without rewriting the security logic. It also makes testing easier because core functions can be tested without opening the graphical application.

I also learned about validation and feedback from the interface. Users need to know when a field is required, when a password is incorrect, when an operation succeeds, and when a value cannot be accepted. The project made me more aware that software quality is not only measured by whether a function works internally. The application must also communicate its state clearly to the person using it.

Testing was one of the most useful parts of the internship because it changed my approach from simply writing code to verifying behaviour. The project contains automated unit tests for encryption, key derivation, password generation, strength checking, CRUD operations, persistence, master-password changes and database input handling. These tests provide repeatable checks that can be run after changes to the code.

I learned the importance of testing security-related negative cases. It is not enough to verify that the correct password unlocks the vault. The application must also reject a wrong password. Similarly, encryption should be tested not only for successful decryption but also for tampered ciphertext. Database tests should include unusual values rather than only normal names and passwords.

Using a temporary database for tests was another practical lesson. Tests should not accidentally modify the real user vault, and separate tests should not depend on one another. This makes the test suite safer and more repeatable. Running the tests through Python's unittest discovery mechanism also introduced me to a simple workflow that can later be integrated into automated development or continuous-integration processes.

The project improved my problem-solving process during debugging. When an application contains several layers, an error in the interface may actually be caused by the database or encryption layer. I learned to isolate the problem by checking one responsibility at a time. For example, if an entry could not be retrieved, I could separately verify whether the record existed, whether the encrypted value was valid, whether the vault was unlocked, and whether the UI was requesting the correct identifier.

This approach reduced the tendency to make multiple changes at once. Instead, I learned to reproduce the problem, identify the smallest failing component, make one controlled change, and run the relevant tests again. This is a useful professional habit because it creates a clearer relationship between a code change and its result.

I also became more comfortable reading documentation. The project depends on Python, SQLite and the cryptography library, so understanding the documented behaviour of these components was necessary. Rather than relying only on examples found elsewhere, I learned to check function names, parameters, expected return values and error behaviour. This improved both my technical confidence and my ability to work independently when a problem was not directly covered in classroom material.

From a software-engineering perspective, the internship taught me the value of modular design and clear responsibilities. The core directory contains separate modules for cryptography, database operations, password generation and management logic, while the UI directory contains the frames, dialogs, application window and theme. This structure is easier to maintain than placing all functionality in one large file.

I also learned that documentation is part of development rather than something added only at the end. While preparing this report, I had to explain why particular technologies were selected, how data moves through the application, how tests are organised, and which

enhancements remain possible. Writing these explanations helped me identify assumptions and understand the system more completely.

Overall, the internship connected several areas that I had previously studied separately: Python programming, databases, graphical interfaces, software testing and basic security concepts. The password manager project provided a practical setting in which these areas had to work together. The experience has given me a clearer understanding of how a small software product can be planned, implemented, tested and documented within a limited development period.

8. Future work scope

Due to the six-week time limitation, the following enhancements were identified but not implemented. They are natural next steps for this project:

- Encrypted export/import of the vault (e.g. as an encrypted JSON or CSV file) for backup and migration between computers.
- Categories, tags and a favourites view for organising a large number of saved accounts.
- A duplicate-password and weak-password report to help users improve their overall password hygiene.
- Checking stored passwords against known data-breach lists using a privacy-preserving API such as Have I Been Pwned's k-anonymity model.
- An optional second factor (e.g. a TOTP-based one-time code) required in addition to the master password to unlock the vault.
- A command-line interface as an alternative to the Tkinter GUI, for scripting and automation use cases.
- Packaging the application as a standalone .exe (e.g. using PyInstaller) so it can be installed without a Python environment.

- The application could support multiple independent vaults. This would allow users to maintain separate collections of credentials for different purposes while keeping each vault protected independently.
- Detailed documentation could be added describing the application's security architecture, encryption workflow, database structure, threat considerations, and recommended usage practices. This would make the project easier for developers and users to understand and maintain.
- The interface could be extended to support multiple languages. This would make the application more accessible to users from different regions.
- A secure activity log could record important vault events such as successful unlocks, failed authentication attempts, password changes, and entry modifications. This could help users monitor activity within their vault.
- An auto-fill feature could be introduced to automatically enter usernames and passwords into supported login forms. This could reduce the need for manually copying credentials from the application.
- Future versions could improve the graphical interface by adding themes, better navigation, keyboard shortcuts, icons, sorting options, and improved accessibility features.
- Future versions could provide more detailed and user-friendly error messages while ensuring that sensitive information is never exposed through error messages or application logs.
- A dashboard could be added to provide an overview of password-related information, such as weak passwords, reused passwords, recently changed passwords, and accounts that require attention.
- An automatic backup mechanism could be introduced so that users can create regular encrypted backups of their vault. This would help protect against accidental deletion, database corruption, or loss of the local computer.

- A future version could provide optional encrypted synchronization between multiple trusted devices. Before synchronization, sensitive vault information should remain encrypted so that the stored data is not exposed during transfer.
- The application could provide reminders when a password has not been changed for a long period. This feature could help users regularly review important account credentials.
- The vault could be extended to securely store other sensitive information such as recovery codes, software licence keys, Wi-Fi credentials, and other private notes. These values would also need to be protected using encryption.
- Detailed documentation could be added describing the application's security architecture, encryption workflow, database structure, threat considerations, and recommended usage practices. This would make the project easier for developers and users to understand and maintain.